ORF307_HW8_soln

ORF307 Homework 8 Solution

Q1

[1]:
```
import matplotlib.pyplot as plt
import cvxpy as cp
import numpy as np
np.random.seed(2)
```

[2]:
```
# Utility functions
def polish(x, tol=1e-03):
    idx = np.isclose(x, np.round(x), rtol=1e-03, atol=1e-04)
    x = np.copy(x)
    x[idx] = np.round(x[idx])
    return x


def is_feasible(x, tol=1e-3):
    """Determines if x is in the feasible set Ax <= b (with tolerance)."""
    if np.any(A @ x - b > tol):
        return False
    return True


def argmin_leaves(nodes, Uglob):
    """Returns the node with lowest lower bound, from a list of nodes.
    """
    m = min([x.lower for x in nodes])
    for x in nodes:
        if x.lower == m:
            return x

def incumbent(Uglob, incumbent, left, right):
    if Uglob == left.upper:
        return left.incumbent
    elif Uglob == right.upper:
        return right.incumbent
    else:
```

```
        return incumbent
```

[3]:
```
# Define node object
class node:
    """Create node of a branch-and-bound tree with size n. Specify the
    optional parent if the node is not at the top of the tree.
    """

    def __init__(self, n, cuts=[], parent=None):
        self.cuts = cuts
        self.parent = parent
        self.left = None
        self.right = None
        self.n = n
        self.alive = True
        self.incumbent = None
        self.x_relaxed = None
        self.lower = np.inf
        self.upper = np.inf

    def depth(self):
        d = 0
        p = self.parent
        while p is not None:
            d += 1
            p = p.parent
        return d

    def solve(self):
        """Find upper and lower bounds of the sub-problem belonging to
        this node."""
        x = cp.Variable(self.n)

        constraints = [A @ x <= b]
        for i, d, f in self.cuts:
            constraints += [d * x[i] <= f]

        problem = cp.Problem(cp.Minimize(c @ x), constraints)
        problem.solve()

        if x.value is None:
            # We couldn't find a solution.
            self.raw = None
            self.lower = np.inf
            self.upper = np.inf
            return np.inf
```

```
        # Use heuristic to choose which variable we should split on next. Don't
        # choose a variable we have already set.
        x_polish = polish(x.value)
        self.x_relaxed = x_polish
        x_round = np.round(x_polish)
        self.picknext = np.argmax(abs(x_polish - x_round))
        self.lower = problem.objective.value

        if is_feasible(x_round):
            self.upper = c @ x_round
            self.incumbent = x_round
        else:
            self.upper = np.inf

        return self.upper

    def addleft(self):
        """Add a node to the left-hand side of the tree and solve."""
        if self.left is None:
            # add cut by rounding relaxation solution
            new_cut = [(self.picknext, 1, np.floor(self.x_relaxed[self.
↪picknext]))]
            self.left = node(self.n, self.cuts + new_cut, self)

        self.left.solve()
        return self.left

    def addright(self):
        """Add a node to the right-hand side of the tree and solve."""
        if self.right is None:
            # add cut by rounding relaxation solution
            new_cut = [(self.picknext, -1, -np.ceil(self.x_relaxed[self.
↪picknext]))]
            self.right = node(self.n, self.cuts + new_cut, self)

        self.right.solve()
        return self.right

    def nodes(self, all=False):
        """Returns a list of all nodes that live at, or below, this point.

        If all is False, return nodes only if they are stil alive.
        """

        coll = []
        if self.alive or all:
            coll += [self]
```

```
        if self.left is not None and (self.left.alive or all):
            coll += self.left.nodes(all)
        if self.right is not None and (self.right.alive or all):
            coll += self.right.nodes(all)

        return coll

    def __repr__(self, level=0):
        """Utility to display bnb tree"""
        ret = "\t"*level+repr(self.depth())
        ret += ' <node: cuts=%d, L=%.2f, U=%.2f>' % (len(self.cuts), self.
↪lower, self.upper)
        if any(self.cuts):
            ret += "\n"
            for i, d, f in self.cuts:
                ret += "\t"*level+"    %.2f x_%d <= %.2f\n" % (d, i, f)
        else:
            ret += "\n"

        if self.incumbent is not None:
            ret += "\t"*level+"  incumbent = " + str(self.incumbent) + "\n"

        if self.x_relaxed is not None:
            ret += "\t"*level+"  x_rel = " + str(self.x_relaxed) + "\n"

        if self.left is not None:
            ret += self.left.__repr__(level+1)
        if self.right is not None:
            ret += self.right.__repr__(level+1)
        return ret
```

[4]:
```
def solve_bb(c, A, b):
    # Initialize history
    uppers = []
    lowers = []
    leavenums = []
    incumbents = []

    # Create the top node in the tree.
    top = node(n)

    Uglob = top.solve()
    incumbents.append(top.incumbent)
    Lglob = -np.inf
    leaves = [top]
    iter = 0
```

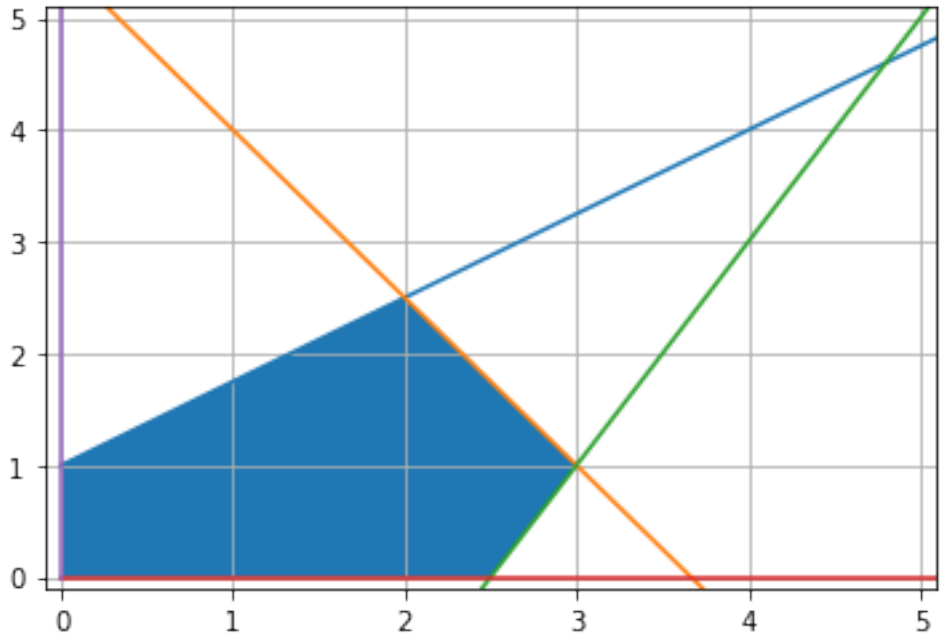
```
    max_iter = 20000
    tol = 1e-03

    # Expand the tree until the gap has disappeared.
    print("Branch and bound iterations")
    while Uglob - Lglob > tol:
        iter += 1
        # Expand the leaf with lowest lower bound.
        l = argmin_leaves(leaves, Uglob)
        left = l.addleft()
        right = l.addright()
        leaves.remove(l)
        leaves += [left, right]

        Lglob = min([x.lower for x in leaves])
        Uglob = min(Uglob, left.upper, right.upper)
        x_bb = incumbent(Uglob, incumbents[-1], left, right)

        # Prune anything except the currently optimal solution. Doesn't
        # actually affect the progress of the algorithm.
        for x in top.nodes():
            if x.lower > Uglob and x.upper != Uglob:
                x.alive = False

        print("iter %3d.  L: %.5f, U: %.0f" %
              (iter, Lglob, Uglob))

        # History
        lowers.append(Lglob)
        uppers.append(Uglob)
        leavenums.append(len([1 for x in leaves if x.alive]))
        incumbents.append(x_bb)

        if iter >= max_iter:
            break

    print("done.")

    return x_bb, top, iter, lowers, uppers, leavenums
```

1.1

[6]:
```
x1 = np.linspace(0,6,10)
x2 = np.linspace(0,6,10)
plt.plot(x1, 1 + 3*x1/4)
plt.plot(x1,11/2 - 3*x1/2)
plt.plot(x1, -5 + 2*x1)
```

```
plt.plot(x1, np.zeros(10))
plt.plot(np.zeros(10), x2)
plt.xticks(np.arange(0,6,1))
plt.yticks(np.arange(0,6,1))
plt.grid()
plt.ylim(-0.1,5.1)
plt.xlim(-0.1,5.1)
x1s = [0,0,2,3,2.5 ]
x2s = [0,1,5/2,1,0]
plt.fill(x1s,x2s)
```

[6]: [<matplotlib.patches.Polygon at 0x170f7fd3400>]



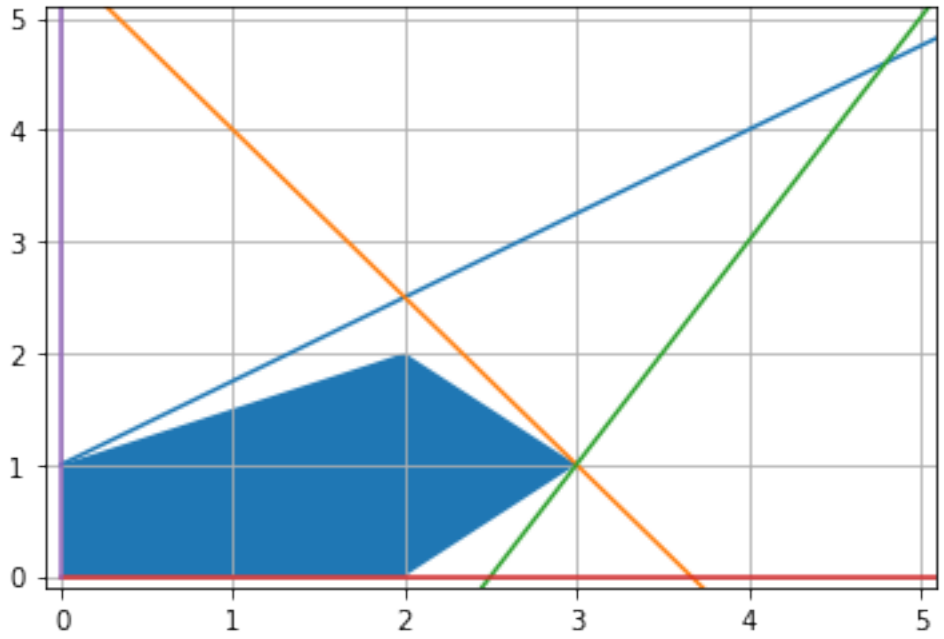
Optimal cost is -7 for the LP and -6 for the IP.

1.2

[7]:
```
x1 = np.linspace(0,6,10)
plt.plot(x1, 1 + 3*x1/4)
plt.plot(x1,11/2 - 3*x1/2)
plt.plot(x1, -5 + 2*x1)
plt.plot(x1, np.zeros(10))
plt.plot(np.zeros(10), x2)
plt.xticks(np.arange(0,6,1))
plt.yticks(np.arange(0,6,1))
plt.grid()
```

```
plt.ylim(-0.1,5.1)
plt.xlim(-0.1,5.1)
x1s = [0,0,2,3,2 ]
x2s = [0,1,2,1,0]
plt.fill(x1s,x2s)
```

[7]: [<matplotlib.patches.Polygon at 0x170f8043f98>]



1.3

[8]:
```
# Double check with the code from the lecture
c = np.array([-1, -2])
A = np.array([[-3, 4],
              [3, 2],
              [2, -1],
             [-1, 0],
             [0, -1]])
b = np.array([4, 11, 5, 0, 0])
n = len(c)

x_bb, tree, iter, lowers, uppers, leavenums = solve_bb(c, A, b)
tree
```

```
Branch and bound iterations
iter   1.  L: -6.33333, U: -6
iter   2.  L: -6.00000, U: -6
done.
```

[8]:
```
0 <node: cuts=0, L=-7.00, U=-6.00>
  incumbent = [2. 2.]
  x_rel = [2.  2.5]
        1 <node: cuts=1, L=-6.33, U=-6.00>
            1.00 x_1 <= 2.00
          incumbent = [2. 2.]
          x_rel = [2.33333333 2.        ]
                2 <node: cuts=2, L=-6.00, U=-6.00>
                    1.00 x_1 <= 2.00
                    1.00 x_0 <= 2.00
                  incumbent = [2. 2.]
                  x_rel = [2. 2.]
                2 <node: cuts=2, L=-5.00, U=-5.00>
                    1.00 x_1 <= 2.00
                    -1.00 x_0 <= -3.00
                  incumbent = [3. 1.]
                  x_rel = [3. 1.]
        1 <node: cuts=1, L=inf, U=inf>
            -1.00 x_1 <= -3.00
```

[9]:
```
tree
```

[9]:
```
0 <node: cuts=0, L=-7.00, U=-6.00>
  incumbent = [2. 2.]
  x_rel = [2.  2.5]
        1 <node: cuts=1, L=-6.33, U=-6.00>
            1.00 x_1 <= 2.00
          incumbent = [2. 2.]
          x_rel = [2.33333333 2.        ]
                2 <node: cuts=2, L=-6.00, U=-6.00>
                    1.00 x_1 <= 2.00
                    1.00 x_0 <= 2.00
                  incumbent = [2. 2.]
                  x_rel = [2. 2.]
                2 <node: cuts=2, L=-5.00, U=-5.00>
                    1.00 x_1 <= 2.00
                    -1.00 x_0 <= -3.00
                  incumbent = [3. 1.]
                  x_rel = [3. 1.]
        1 <node: cuts=1, L=inf, U=inf>
            -1.00 x_1 <= -3.00
```

Q2

2.1 Define the binary variables x and y as follows:

$$x_i = \begin{cases} 1 \text{ if box } i \text{ is used,} \\ 0 \text{ otherwise.} \end{cases}$$

$$y_{ij} = \begin{cases} 1 \text{ if item } j \text{ is assigned to box } i, \\ 0 \text{ otherwise.} \end{cases}$$

Then we can formulate our integer program as:

$$\begin{array}{ll} \text{minimize} & 0 \\ \text{subject to} & b^T x \leq Q \\ & \sum_{j=1}^{n} a_j y_{ij} \leq b_i, \quad \text{for } i = 1, \ldots, m \\ & \sum_{i=1}^{m} y_{ij} = 1, \quad \text{for } j = 1, \ldots, n \\ & y_{ij} \leq x_i, \quad \text{for } i = 1, \ldots, m, j = 1, \ldots, n \end{array}$$

2.2

[10]:
```
n = 30 # Number of items
m = 10 # Number of boxes
Q = 100 # Truck capacity

# Item sizes
a = np.array([4, 3, 1, 2, 2,
              5, 4, 2, 4, 2,
              4, 1, 5, 2, 1,
              3, 5, 3, 3, 3,
              4, 5, 5, 2, 3,
              3, 3, 2, 2, 2])

# Box sizes
b = np.array([7, 19, 10, 14, 10, 12, 13, 10, 15, 14])
```

[11]:
```
x = cp.Variable(m, boolean=True)
y = cp.Variable((m, n), boolean=True)

objective = cp.Minimize(cp.sum(x))
constraints = [b @ x <= Q]
constraints += [a @ y[i, :] <= b[i] for i in range(m)]
constraints += [cp.sum(y, axis=0) == 1]
constraints += [y[:, j] <= x for j in range(n)]
problem = cp.Problem(objective, constraints)
problem.solve(solver=cp.SCIPY)
```

[11]:
```
7.0
```

[12]:
```
print("Total number of boxes =", x.value)
```

```
Total number of boxes = [0. 1. 1. 1. 0. 1. 0. 1. 1. 1.]
```